BioSpecInfo

Riferimento delle interfacce — BioSpecInfo

Samuele Pio Provenzano

Versione **bsi-v247** · commit `0a2ef80`

Generato il 2026-11-11

Documento generato automaticamente dal sorgente Markdown versionato nel repository. La versione e il commit stampati in ogni piè di pagina identificano esattamente lo stato del software descritto.

Riferimento delle interfacce — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v247`
Scopo	Documentare le interfacce HTTP del progetto e le API esterne consumate dall'applicazione.

0. Premessa necessaria

BioSpecInfo non espone API. È una Progressive Web App interamente client-side, servita come file statici: non esiste un backend, non esiste un server applicativo, non esistono endpoint REST del prodotto.

Questo documento sarebbe quindi vuoto, se non fosse per due interfacce che esistono davvero e che un valutatore ha diritto di vedere documentate:

	Cosa è	Documentato in
Proxy Spectra	Un Cloudflare Worker **opzionale**, di proprietà dell'utente, che inoltra le chiamate ai fornitori AI. È l'unica interfaccia HTTP scritta in questo progetto.	§1, §2
API esterne consumate	Servizi pubblici di terzi che l'applicazione interroga, sempre in modo opzionale.	§3
API interne JavaScript	I punti di estensione del codice, per chi deve intervenirci.	§4

> Una specifica OpenAPI di endpoint inesistenti sarebbe documentazione plausibile e falsa. Qui è descritto soltanto ciò che esiste.

1. Proxy Spectra — panoramica

Il proxy risolve due problemi che dal browser non hanno altra soluzione:

1. **CORS.** Diversi fornitori AI non inviano le intestazioni che permettono a

una pagina web di chiamarli direttamente. Dal Worker la chiamata parte da un server e il vincolo non si applica.

1. **Custodia delle chiavi.** Con il proxy la chiave API risiede fra i segreti

del Worker e non sul dispositivo dell'utente.

È **opzionale**: senza proxy l'applicazione funziona usando le chiavi inserite localmente dall'utente.

Deploy

```
cd proxy
npx wrangler login
npx wrangler deploy
npx wrangler secret put GROQ_KEYS     # una o più chiavi, separate da virgola
```

Configurazione

Variabile	Tipo	Default	Descrizione
`ORIGINI`	segreto/variabile	*(vuoto = tutte)*	Elenco di origini ammesse, separate da virgola. **Lasciarlo vuoto rende il proxy aperto a chiunque:** sconsigliato.
`LIMITE_IP`	variabile	`20`	Richieste per singolo IP nella finestra temporale
`TETTO_GIORNO`	variabile	`2000`	Richieste totali giornaliere
`GROQ_KEYS`, `GEMINI_KEYS`, `ANTHROPIC_KEYS`, `OPENAI_KEYS`, `XAI_KEYS`, `NVIDIA_KEYS`, `ZAI_KEYS`, `DEEPSEEK_KEYS`	segreti	—	Chiavi API. Più chiavi separate da virgola: esaurita la quota della prima (429), il proxy passa alla successiva **nella stessa richiesta**.

Endpoint

Metodo	Percorso	Descrizione
`OPTIONS`	`/*`	Preflight CORS
`GET`	`/stato`	Quali fornitori hanno una chiave configurata
`GET` `POST`	`/{fornitore}/{percorso...}`	Inoltro al fornitore

Fornitori riconosciuti: `anthropic` · `gemini` · `groq` · `xai` · `nvidia` · `zai` · `openai` · `deepseek`

Codici di stato

Codice	Significato
`200`	Risposta del fornitore, inoltrata così com'è
`204`	Preflight CORS accettato
`400`	Corpo della richiesta illeggibile
`403`	Origine non ammessa dall'elenco `ORIGINI`
`404`	Fornitore sconosciuto
`405`	Metodo diverso da GET, POST o OPTIONS
`429`	Limite per IP o tetto giornaliero superato

Codice	Significato
`503`	Nessuna chiave configurata per quel fornitore

Scelte di sicurezza

- **Nessuna intestazione inoltrata ciecamente.** Passano solo

`anthropic-version` e `anthropic-beta`; qualunque credenziale inviata dal client resta fuori. Un proxy che inoltra tutto è un relay aperto.

- **Il parametro `key` non viene propagato.** Se un client lo mette in query

string, il proxy lo scarta: la chiave è quella del server.

- **Il corpo viene letto una volta sola e conservato**, perché per ritentare con

una chiave diversa occorre poterlo rimandare — e uno stream si consuma.

---

2. Specifica OpenAPI 3.0

```yaml
openapi: 3.0.3
info:
  title: Proxy Spectra
  version: "1.0.0"
  description: |
    Proxy opzionale (Cloudflare Worker) per le chiamate ai fornitori di modelli
    linguistici. Elimina i blocchi CORS e mantiene le chiavi API sul server
    anziché sul dispositivo dell'utente.

    Non fa parte dell'applicazione BioSpecInfo: viene pubblicato dall'utente sul
    proprio account Cloudflare, e l'applicazione lo usa solo se ne conosce
    l'indirizzo.
  license:
    name: Proprietary — All rights reserved (Samuele Pio Provenzano)
servers:
  - url: https://{nomeworker}.{account}.workers.dev
    description: Istanza dell'utente
    variables:
      nomeworker: { default: spectra-proxy }
      account:    { default: esempio }

tags:
  - name: Stato
    description: Interrogazione della configurazione, senza rivelare segreti
  - name: Inoltro
    description: Inoltro delle richieste ai fornitori di modelli

paths:
  /stato:
    get:
      tags: [Stato]
      summary: Fornitori configurati
      description: |
        Elenca i fornitori per i quali il proxy possiede almeno una chiave.
        Non rivela alcuna chiave. L'applicazione lo usa per mostrare soltanto i
        modelli che funzionano davvero, invece di farli fallire uno per uno.
      responses:
        "200":
          description: Elenco dei fornitori attivi
          content:
            application/json:
              schema:
                type: object
                required: [fornitori]
                properties:
                  fornitori:
                    type: array
                    items:
                      type: string
                      enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
              examples:
                dueChiavi:
                  summary: Due fornitori configurati
                  value: { fornitori: [groq, gemini] }
                nessuna:
                  summary: Worker pubblicato ma senza segreti
                  value: { fornitori: [] }
```

```
      "403":
        $ref: "#/components/responses/OrigineNonAmmessa"

  /{fornitore}/{percorso}:
    parameters:
      - name: fornitore
        in: path
        required: true
        description: Fornitore di destinazione
        schema:
          type: string
          enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
      - name: percorso
        in: path
        required: true
        description: |
          Percorso dell'endpoint presso il fornitore, ricomposto così com'è.
          Esempi: `openai/v1/chat/completions` per Groq,
          `v1beta/models/gemini-3.6-flash:streamGenerateContent` per Gemini.
        schema: { type: string }
        example: openai/v1/chat/completions
    post:
      tags: [Inoltro]
      summary: Inoltra una richiesta al fornitore
      description: |
        Il corpo viene trasmesso invariato. Il proxy aggiunge la chiave nel modo
        richiesto da quel fornitore (intestazione `Authorization: Bearer`,
        `x-api-key`, oppure parametro in query) e, se la chiamata fallisce per
        quota esaurita, ritenta con la chiave successiva nella stessa richiesta.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              description: Corpo previsto dall'API del fornitore, trasmesso invariato
            examples:
              groq:
                summary: Completamento in streaming su Groq
                value:
                  model: llama-3.3-70b-versatile
                  stream: true
                  messages:
                    - role: user
                      content: Quali bande IR identificano un acido carbossilico?
      responses:
        "200":
          description: |
            Risposta del fornitore, inoltrata senza modifiche (tipicamente
            `text/event-stream` quando `stream: true`).
          content:
            application/json:
              schema: { type: object }
            text/event-stream:
              schema: { type: string }
        "400": { $ref: "#/components/responses/CorpoIlleggibile" }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }
```

```
    get:
      tags: [Inoltro]
      summary: Inoltra una richiesta in lettura al fornitore
      description: |
        Serve, fra l'altro, alla risoluzione del modello Gemini, che interroga
        `ListModels` in GET e senza chiave nel browser non potrebbe passare.
      responses:
        "200":
          description: Risposta del fornitore
          content:
            application/json:
              schema: { type: object }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }

components:
  schemas:
    Errore:
      type: object
      required: [errore]
      properties:
        errore:
          type: string
          description: Messaggio in italiano, pensato per essere mostrato all'utente
  responses:
    OrigineNonAmmessa:
      description: L'origine della richiesta non è nell'elenco `ORIGINI`
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Origine non ammessa da questo proxy." }
    FornitoreSconosciuto:
      description: Il primo segmento del percorso non corrisponde a un fornitore noto
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Fornitore sconosciuto: pippo" }
    CorpoIlleggibile:
      description: Il corpo della richiesta non è stato leggibile
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Corpo della richiesta illeggibile." }
    TroppeRichieste:
      description: Superato il limite per IP o il tetto giornaliero
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Troppe richieste da questo indirizzo: riprova fra un minuto." }
    ChiaveAssente:
      description: Il proxy non ha una chiave configurata per quel fornitore
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example:
            errore: "Il proxy non ha una chiave per groq. Impostala con: wrangler secret put
GROQ_KEYS"
```

```
security: []
# L'autenticazione del CLIENT è volutamente assente: il controllo di accesso è
# l'elenco ORIGINI più i limiti per IP. Aggiungere un token lato client
# significherebbe scriverlo in una pagina statica, cioè renderlo pubblico.
```

3. API esterne consumate dall'applicazione

Tutte **opzionali**: senza rete l'applicazione resta pienamente funzionante sui dati locali, e lo dichiara all'utente.

Servizio	Endpoint usati	A cosa serve	Se non risponde

PubChem PUG-REST	`/compound/name/{nome}/property/...`
	 `/compound/smiles/{smiles}/SDF?record_type=3d`
	 `/compound/cid/{cid}/PNG`
		Nome IUPAC, formula, CAS, pittogrammi GHS, conformeri 3D, immagini 2D	L'app usa i dati tabulati locali e lo segnala

RCSB PDB	`https://files.rcsb.org/download/{pdb}.pdb`	Strutture proteiche per il visore 3D	Messaggio esplicito nel visore
NASA / ESA	Immagini e dati pubblici	Modulo di astrochimica	Il modulo mostra i dati tabulati
Fornitori AI	A scelta dell'utente, o via proxy	Risposte dell'assistente	L'assistente lo dichiara e propone un'alternativa raggiungibile
Crossref REST	`https://api.crossref.org/works/{doi}`	Scheda di un'opera per le citazioni (APA 7, IEEE, ISO 690, BibTeX) nelle sezioni Risorse e bibliografia	Lo dichiara, con l'indirizzo della richiesta; resta il modulo da compilare a mano
Stesso sito dell'app	`./sw.js?controllo=<ora>` con `cache: 'no-store'`	«Controlla aggiornamenti»: la riga `var CACHE = 'bsi-vNNN'` è la versione pubblicata	Si prova il ripiego su GitHub
GitHub (ripiego)	`https://raw.githubusercontent.com/samupropio1-ship-it/BioSpecInfo-v11/main/sw.js`	Stessa lettura, solo se lo stesso sito non risponde	«Impossibile verificare ora», l'app continua offline

Nessuna di queste chiamate trasmette dati personali. Non sono presenti identificativi persistenti né telemetria attiva (`BSI_TELEMETRY_URL` è vuoto).

4. API interne JavaScript

Punti di estensione per chi deve intervenire sul codice. Sono globali sul `window` perché l'applicazione è un insieme di script classici, non moduli ES.

`window.BSISpettri` — motore spettrale

Funzione	Firma	Descrizione
`gruppi`	`(smiles, callback)`	Riconosce i gruppi funzionali via SMARTS/RDKit. Richiama con `null` se RDKit non è disponibile: il chiamante decide se ripiegare o dichiarare che non può calcolare.
`gruppiSync`	`(smiles) → oggetto | null`	Versione sincrona: risponde solo se la molecola è già in cache
`bandeDaGruppi`	`(gruppi) → banda[]`	Tabella delle bande IR `{n, pk, w, i, f}`
`svgIR`	`(bande, opzioni) → string`	Spettro IR in trasmittanza, SVG, deterministico
`legendaIR`	`(bande) → string`	Assegnazioni in HTML
`quandoPronto`	`(callback)`	Notifica quando l'analisi di una molecola è disponibile, per ridisegnare
`precarica`	`()`	Scalda RDKit senza che nessuno stia aspettando

`window.bsiProxy` — stato del proxy

Funzione	Descrizione
`url()`	Indirizzo configurato, stringa vuota se assente
`stato()`	Promessa: interroga `/stato` e memorizza i fornitori coperti
`copre(idFornitore)`	Se quel fornitore è coperto dal proxy
`ricarica()`	Rilegge la configurazione

Altri punti notevoli

Simbolo	Descrizione
`window.bsiLoadRDKit(cb)`	Carica RDKit una volta sola e richiama con l'istanza
`window.irBandList(smiles)`	Bande IR — sostituita dal motore nuovo, mantiene la firma originale
`window.makeIRsvg(bande, colore)`	Disegno IR — idem
`window.nmrPeakList(smiles)`	Picchi ^{1}H con integrazione
`window.bsiCancellaDati(gruppi)`	Cancellazione selettiva dei dati salvati
`window.bsiOpenUpdate()`	Apre la finestra degli aggiornamenti

`window.BSIAggiorna` — controllo degli aggiornamenti (`bsi-aggiorna.js`)

Funzione	Firma	Descrizione
`controlla`	`() → Promise<'aggiornata' | 'nuova' | 'offline'>`	Confronta la versione in uso con quella pubblicata e lo dice in un avviso a comparsa (`#bsi-agg-esito`). Se la versione è diversa chiede al service worker di aggiornarsi (`registration.update()`). Risponde sempre
`versioneInUso`	`() → Promise<string | null>`	Chiede al service worker attivo il nome della sua cache con il messaggio `{ type: 'BSI_VERSIONE' }` su un `MessageChannel`; `null` senza service worker o dopo 1,5 s senza risposta
`versionePubblicata`	`() → Promise<string | null>`	Scarica `./sw.js?controllo=<ora>` senza cache e ne legge `var CACHE = '…'`

Il service worker risponde al messaggio `BSI_VERSIONE` con `{ cache: CACHE }` sulla porta ricevuta. La finestra 🔄 Aggiornamenti usa queste funzioni e, se lo stesso sito non risponde, legge la copia di `sw.js` su GitHub.

`window.BSIAttrezzi` — strumenti rapidi del menu ✨ (`bsi-attrezzi.js`)

Simbolo	Firma	Descrizione
`apri`	`(chiave, materia) → boolean`	Apre lo strumento in una finestra sopra la pagina. Chiavi: `calcolatrice`, `derivata`, `grafico`, `costante`, `unita`, `formula`, `glossario`, `quiz`. `false` se la chiave non esiste
`chiudi`	`()`	Chiude la finestra
`STRUMENTI`	oggetto	Icona, nome, descrizione e funzione di apertura di ogni strumento
`PER_MATERIA`	oggetto	Quali strumenti mostra il menu ✨ per `fisica` e `matematica` (per `chimica` nessuno)
`calcolo.calcola`	`(testo) → { esatto, html, valore } | { errore }`	Calcolo esatto con `BSIMatCAS`
`calcolo.derivaTesto`, `calcolo.integraTesto`	`(testo, x?) → { testo, html } | { errore }`	Derivata e primitiva simboliche
`calcolo.campioni`	`(testo, a, b, n)`	Punti di una funzione di x per il grafico; `null` senza motore simbolico
`calcolo.cerca`	`(lista, q, campi)`	Ricerca che non bada agli accenti
`calcolo.quizLampo`	`(materia, n, seme)`	Domande estratte dal kit di studio con un generatore riproducibile; `null` senza kit

`window.BSIStudio` — motore dei kit di studio (`bsi-studio.js`)

Simbolo	Firma	Descrizione
`registra`	`(materia, dati) → boolean`	Unisce i dati di una materia (`argomenti`, `riassunti`, `quiz`, `flash`, `formule`, `glossario`) e crea le cinque sezioni `s<pre>riass`, `s<pre>quiz`, `s<pre>flash`, `s<pre>form`, `s<pre>gloss` (`pre`: `fis`, `mat`, `chi`). Un file di dati caricato prima del motore si mette in coda in `window.BSIStudioCoda`
`dati`	`(materia) → oggetto | null`	I dati uniti della materia
`materie`	`() → string[]`	Le materie registrate
`sezioni`	`(materia) → string[]`	Gli id delle cinque sezioni
`valida`	`(dati, opz) → string[]`	I problemi trovati, in italiano; un elenco vuoto vuol dire «va bene». Con `{ quantita: true }` controlla anche i minimi di `MINIMI`
`MINIMI`	oggetto	`{ riassunti: 60, quiz: 300, flash: 250, formule: 200, glossario: 250 }`
`pulisci`	`(html) → string`	Tiene solo `p, b, i, ul, li, sub, sup, br`, senza attributi
`norm`	`(s) → string`	Testo senza tag e senza accenti, minuscolo
`calcolo.mulberry32`	`(seme) → () → number`	Generatore riproducibile in [0, 1)
`calcolo.mescola`	`(arr, rnd?) → array`	Fisher–Yates su una copia
`calcolo.leitner`	`(stato, id, giusta) → stato`	Giusta: una scatola su (massimo 5); sbagliata: scatola 1. Restituisce un oggetto nuovo
`calcolo.punteggio`	`(risposte) → { giuste, totale, perc, perArg }`	Punteggio complessivo e per argomento

`window.BSISpectraAndroide` — il volto di Spectra (`bsi-spectra-androide.js`)

Simbolo	Firma	Descrizione
`disponibile`	`() → boolean`	Se il browser offre WebGL
`crea`	`(contenitore) → Promise<Androide>`	Carica three.js, `GLTFLoader` e `models/spectra-noir.glb`, e disegna il volto nel contenitore
`foto`	`(px) → Promise<string>`	Un'immagine PNG del viso (data URL), per le icone
`emozioneDi`	`(frase) → string`	L'emozione di una frase dalle parole chiave; `'serena'` senza segnali
`EMOZIONI`	oggetto	Le 17 emozioni, ognuna come pesi dei campi espressivi più luce degli occhi e posa della testa
`CAMPI`	string[]	Gli 11 campi espressivi
`COLORI`	oggetto	Colore degli occhi per stato; `attesa: 0x1a3050`

Simbolo	Firma	Descrizione
`URL`, `BOCCA`, `versione`	—	Percorso del modello, geometria della bocca, versione del modulo

L'oggetto restituito da `crea` ha, fra gli altri, `stato(s)` (`'attesa'`, `'ascolta'`, `'pensa'`, `'parla'`), `dici(testo)` (la bocca segue il testo), `allinea(i)`, `taci()`, `espressione(nome, secondi, intensita) → boolean`, `emozione() → string`, `sente(frase)` (reagisce a una frase dell'utente e la passa alla mente), `passo(dt)`, `ferma()`, `libera()`. Dalla versione 3.0 del modulo (v238) anche `concentra(c)` (concentrazione bersaglio fra 0 e 1), `concentrazione() → number` (quella del momento), `addormentato() → number` (0 sveglio, 1 addormentato), `aperture() → [sx, dx]` (apertura visibile delle due palpebre, 1 = a riposo) e la proprietà `coloreOcchi` (tinta corrente degli occhi, `#rrggbb`).

`window.BSISpectraRobot` (`bsi-spectra-robot.js`) espone `disponibile`, `crea` e `foto` con le stesse firme: usa l'androide e, se il modello non arriva, la testa costruita in three.js. `window.BSISpectraVolto` (`bsi-spectra-volto.js`) è il ritratto SVG di ultimo ripiego.

`window.BSISpectraMente` — la mente di Spectra (`bsi-spectra-mente.js`)

Funzione	Firma	Descrizione
`ricorda`	`(domanda, esito)`	Registra un episodio (ora, materia, sezione, domanda troncata a 160 caratteri) e aggiorna curiosità, attenzione e legame
`sente`	`(emozione, chi)`	Sposta l'umore (e la premura) secondo l'emozione rilevata in una frase
`stato`	`() → { curiosita, attenzione, premura, umore, legame }`	Lo stato interno, valori fra 0 e 1
`fondo`	`() → string`	L'emozione di fondo del volto quando nessuno parla
`profilo`	`() → oggetto`	Sezioni preferite, minuti per materia, ora preferita, materia trascurata, errori dei quiz, ultimi argomenti
`pensieri`	`() → [{ peso, testo }]`	Suggerimenti ordinati per peso
`saluto`	`() → string`	Saluto che tiene conto dell'ora e dell'ultima visita
`coscienza`	`() → string`	Il quadro che si aggiunge alla domanda inviata al modello; vuoto senza memoria. Esposto anche come `window._bsiMente`
`cosaRicordi`, `comeStai`	`() → string`	Le risposte ai comandi «cosa ricordi di me» e «come stai»
`dimentica`	`()`	Azzera la memoria e rimuove la chiave `bsi-spectra-mente-v1`

È una simulazione di comportamento: le funzioni leggono e scrivono numeri e testi nel `localStorage`, nient'altro.

`window.BSISpectraLive` — la console (`bsi-spectra-live.js`), aggiunte della v238

Funzione	Firma	Descrizione
`difficolta`	`(testo) → { valore, livello, nome }`	Difficoltà della domanda fra 0,05 e 1, livello 0–3 e il suo nome (scuola superiore … dottorato)
`livelloSezione`	`(id) → number`	Primo livello del percorso di `bsi-percorsi.js` in cui compare la sezione; −1 se in nessuno
`navigatore`	`(materia?)`	Apre (o chiude) il navigatore 🧭; con una materia, mostra quella
`provaCreatore`	`(attiva)`	Mette il campo della console in modalità prova (tipo password per un minuto) o la toglie
`verificaCreatore`	`(frase)`	Confronta la frase con `BSISpectraCreatore.verifica` e risponde nel dialogo, senza registrare la frase

`window.BSISpectraCreatore` — il creatore (`bsi-spectra-creatore.js`)

Simbolo	Firma	Descrizione
`NOME`, `NOME_BREVE`	string	`'Samuele Pio Provenzano'`, `'Samuele'`
`riconosciuto`	`() → boolean`	Se su questo dispositivo il creatore è stato riconosciuto
`verifica`	`(frase) → Promise<{ ok, rimasti?, attendi? }>`	Impronta SHA-256 della frase confrontata con quella del File Manager; dopo tre errori `attendi` (secondi) per un minuto. La frase non viene conservata
`dimentica`	`()`	Rimuove `bsi-spectra-creatore-v1`
`visita`	`()`	Aggiorna la data dell'ultima visita del creatore riconosciuto
`saluto`, `chiTiHaCreato`	`() → string`	Il saluto per nome e la risposta a «chi ti ha creato?»
`contesto`	`() → string`	La riga `[CREATORE]` aggiunta al quadro per il modello

`window.BSILezione` — la lezione guidata (`bsi-lezione.js`, v239)

Simbolo	Firma	Descrizione
`avvia`	`(sezione?, { muta? }?) → boolean`	Costruisce la lezione della sezione indicata (o di quella visibile) e la avvia nel lettore; `muta: true` parte con la voce spenta. `false` se non c'è una sezione visibile. Una lezione già in corso viene chiusa
`chiudi`	`()`	Ferma la lezione, rimette com'era il comando in dimostrazione, toglie pannello e riflettore, rende di nuovo visibili i pulsanti flottanti
`costruisci`	`(sezione?) → { sezione, titolo, capitoli, passi } | null`	Solo la costruzione, senza lettore. `capitoli`: «Introduzione», un nome per scheda (o «Esploriamo»), «Verifica» se c'è la domanda, «Riepilogo». `passi`: oggetti con `tipo` fra `titolo`, `scheda`, `leggi` (segnaposto sostituito dai passi della scheda quando la si apre), `testo`, `tela`, `cursore`, `scelta`, `quiz`, `fine`, e `cap` (indice del capitolo)
`vai`	`(i)`	Va al passo `i` come un tocco su ◀ ▶ (interrompe la voce)

Simbolo	Firma	Descrizione
`stato`	`() → { i, passi, tipo, capitoli, auto } | null`	Passo corrente, numero di passi, tipo del passo, capitoli, avanzamento automatico attivo; `null` senza lezione
`versione`	string	`'1.0'`

Il comando locale «fammi lezione» di `BSISpectraLive.comando` chiama `BSILezione.avvia()` dopo aver chiuso la console.

`window.BSISpectraVoce` — la voce (`bsi-spectra-voce.js`, v239)

Simbolo	Firma	Descrizione
`qualita`	`() → 'audio' | 'sistema' | 'testo'`	`'audio'` con una chiave OpenAI in `bsi_api_keys` e Web Audio; `'sistema'` con la sola `speechSynthesis`; `'testo'` senza voce
`parla`	`(testo, { suBocca?, timbro?, soloSistema? }?) → Promise<{ ok, motore, durata? } | { annullato }>`	In modalità audio invia il testo (fino a 3 800 caratteri) a `api.openai.com/v1/audio/speech` (`gpt-4o-mini-tts`, voce «onyx»; `tts-1` come ripiego), lo fa passare nella catena e chiama `suBocca({ energia, centroide, apertura, rotondo, largo, frazione })` a ogni fotogramma; se la richiesta fallisce, o con `soloSistema`, usa `speechSynthesis`. Interrompe la frase precedente
`ferma`	`()`	Ferma l'audio e la sintesi del browser
`catena`	`(audioContext, timbro?) → { ingresso, uscita, analizzatore, ferma }`	La catena di effetti del `TIMBRO`
`impulso`	`(audioContext, secondi) → AudioBuffer`	Risposta all'impulso sintetica del riverbero (stereo)
`leggiBocca`	`(analizzatore, dati) → { energia, centroide, apertura, rotondo, largo }`	Labiale dall'audio: energia RMS e centroide spettrale
`TIMBRO`	oggetto	`bassiHz` 140, `bassiDb` 5, `fangoHz` 350, `fangoDb` −3, `presenzaHz` 2800, `presenzaDb` 3, `anelloHz` 55, `anello` 0,12, `pettineMs` 4,3, `pettineRetro` 0,35, `pettine` 0,18, `calore` 1,6, `stanzaS` 1,4, `stanza` 0,16, `volume` 0,95

`window.BSISpectraCorpo` — il corpo (`bsi-spectra-corpo.js`, v239)

Simbolo	Firma	Descrizione
`crea`	`(radice, { animazioni?, capelliNeri? }?) → Corpo`	Collega il driver a un oggetto three.js con scheletro; con `animazioni` (le clip del GLB) riconosce `idle`, `walk`, `sit`, `stand`, `wave`, `point`, `talk`; i capelli diventano neri salvo `capelliNeri: false`
`manichino`	`({ neon? }?) → THREE.Group`	Il manichino di prova (dalla v240 solo per il banco del driver, mai come Spectra): ossa con nomi Mixamo in posa a T, testa con 26 blendshape ARKit, occhi separati, calotta di capelli rossa

Simbolo	Firma	Descrizione
`EMOZIONI`, `OSSA`, `normaNome`	oggetti, funzione	Pesi ARKit per emozione; espressioni regolari dei nomi delle ossa; normalizzazione dei nomi

Metodi di un `Corpo`:

Metodo	Firma	Descrizione
`rapporto`	`() → { ossa, mancanti, blendshape, capelli, clip, locomozione, viso }`	Che cosa il modello sa fare (il controllo dei requisiti di `docs/spectra/01`)
`capelliNeri`	`(colore?)`	Colore (predefinito `#0a0a0c`), rugosità, e texture desaturata nello shader
`orienta`	`(osso, direzione, peso)`	Porta la direzione osso → figlio su una direzione del mondo
`emozione`	`(nome, secondi?, intensita?) → boolean`	Una delle emozioni di `EMOZIONI` per qualche secondo
`dici`, `audio`, `taci`	`(testo)`, `(statoBocca)`, `()`	Labiale dal testo, dall'audio (`suBocca` di `BSISpectraVoce`), stop
`guarda`	`(punto)`	Occhi, collo e testa verso un punto
`fai`	`(gesto, bersaglio?, secondi?)`	`'indica'`, `'spiega'`, `'pensa'`, `'attende'`, `'conclude'`
`cammina`, `siedi`	`(punto)`, `(si?)`	Passo verso un punto; seduta (`false` per alzarsi)
`peso`, `imposta`	`(nome) → number | null`, `(nome, valore) → boolean`	Legge e scrive il peso di un blendshape
`passo`	`(dt)`	Avanza il comportamento di `dt` secondi; va chiamato a ogni fotogramma

`window.BSISpectraFigura` — Spectra col corpo (`bsi-spectra-figura.js`, v240; corpo incorporato dalla v241)

Simbolo	Firma	Descrizione
`Spectra`	`new (T, scena, renderer, opz?) → controllore`	Il corpo di Spectra in una scena esistente. Dalla v241 il corpo è **incorporato**: `carica()` scarica `models/spectra-corpo.bsi`, lo riapre (funzione interna `apri`: firma «BSIC», versione, lunghezza, byte mescolati con un flusso xorshift32 da un seme fisso) e lo passa a `daBuffer`; prima cancella un vecchio corpo dall'IndexedDB `bsi-spectra-modelli`, se c'è. Se il blocco non arriva: con `opz.soloDispositivo` (attivo per la figura di `monta`) nessun altro tentativo e `caricato = false`; senza (lo studio, in sviluppo) prova `models/spectra-corpo.glb`. Il ripiego `models/spectra-cyberpunk.glb` non c'è più. Nessun corpo sostitutivo (il manichino solo con `opz.manichino === true`, per il banco del driver). Rig con `bsi-spectra-rig.js` (`RIG`), volto «Noir» sul colletto, «solidale»

Simbolo	Firma	Descrizione
`monta`	`(contenitore, opz?) → figura`	Riquadro da laboratorio con Spectra in piedi, HUD «SPECTRA · IN ASCOLTO / PENSA / PARLA», pulsanti «🎙️ Parla con me», «🧠 Spiega», «🎓 Studio» (dalla v241 niente «🧍 Corpo» né campo file); mentre il corpo arriva «Spectra sta arrivando…», se non arriva «Spectra non è arrivato: serve la rete la prima volta» con «↻ Riprova»; ascolta `bsi-spectra-stato`, `bsi-spectra-risposta`, `bsi-spectra-dice`
`sezioni`	`(id[]) → figura`	Una figura condivisa (un contesto WebGL) spostata nella sezione dell'AI aperta (`sai`, `sareachim`, `sareafis`, `sareamat`; dalla v242 anche `sesplchim`, `sesplfis`, `sesplmat`, dove la figura prende la classe `bsiEX-palco` e resta in scena mentre si scorre); i file 3D si caricano alla prima apertura. **Dalla v242 la console Spectra Live aperta ha la precedenza su tutto**: la figura va in `#bsiJV-figura` (creato se manca) e la console prende la classe `con-figura`, che nasconde il busto `#bsiJV-volto`; chiusa la console, la figura torna nella sezione
`prepara`	`() → Promise`	Carica three.js, il caricatore GLB, il driver del corpo e il rig se non ci sono
`dbCorpo`	`('leggi' | 'metti' | 'togli', buffer?) → Promise`	L'IndexedDB del vecchio corpo caricato dall'utente (`bsi-spectra-modelli`, archivio `file`, chiave `corpo`); dalla v241 `carica()` la usa solo per leggere e **togliere** un vecchio corpo, non ci scrive più
`emozioneDi`	`(testo) → string`	L'emozione suggerita dal contenuto di una risposta
`Ologramma`	`new (T, scena) → ologramma`	Lo schermo olografico alla destra di Spectra; metodi `costruisci(nome)`, `mostra(nome)`, `passo(dt)`; scene calcolate a ogni fotogramma, più `'schermo'` (la sezione dietro la finestra, ridisegnata con `BSISpectraLive.cattura` 5 volte al secondo; una sezione senza grafici si proietta come scheda: titolo, scheda, fino a 6 comandi con i valori, testo visibile)
`SCENE_OLO`	oggetto	Le sei scene (`pianeti`, `molecola`, `spettro`, `superficie`, `orbitale`, `onda`): per ognuna `nome` (it, en), `sez` (l'id della sezione che apre l'etichetta) e `parole` (l'espressione regolare che la sceglie)
`sceneDi`	`(testo) → string | null`	La scena che corrisponde alle parole di un testo, o `null`
`LIMITE`, `RIG`, `figura`	numero, oggetto, figura	60 MB; i parametri del rig del corpo dell'autore; la figura condivisa delle sezioni (o `null`)

Metodi del controllore: `carica() → Promise<boolean>` (il corpo incorporato; imposta `caricato`), `daBuffer(buffer)`, `usaScena(scena)` (per i banchi), `passo(dt)`, `emozione(nome, secondi?)`, `dici(testo)`, `taci()`, `fai(gesto, bersaglio?, secondi?)`, `presente() → boolean`. `daFile(file)` è stato tolto nella v241 insieme al pulsante «🧍 Corpo».

Evento nuovo: **`bsi-spectra-dice`** (`CustomEvent` su `document`, `detail: { testo }`), emesso da `parla()` di `bsi-spectra-live.js` e da `bsi-assistente.js` quando risolve un esercizio; la figura lo dice con labiale e gesto.

Evento nuovo: **`bsi-spectra-tema`** (`CustomEvent` su `document` , `detail: { testo, schermo }`), emesso da `bsi-spectra-spiega.js` con la domanda (e all'apertura della scheda con una sezione dietro); l'ologramma sceglie la scena con `sceneDi(testo)` , oppure mostra lo schermo dal vivo se `schermo` è vero. Quando risponde, la figura indica prima l'ologramma (gesto «indica», circa 2,5 s) e poi accompagna con «spiega».

`window.BSISpiega` — «🧠 Spiega» (`bsi-spectra-spiega.js` , v240)

Simbolo	Firma	Descrizione
`costruisci`	`(pannello)`	Costruisce la scheda «🧠 Spiega» nel pannello dato (la chiama `bsi-ai-hub.js`): domanda, livello, casella «📺 Includi lo schermo» con la scheda dello schermo, domande rapide, «↺ Nuova spiegazione», il riquadro «🗂️ Tutte le sezioni nell'ologramma» (ricerca, filtro per materia, «🎞️ Mostrami tutto»)
`catalogo`	`() → [{ id, nome, gruppo, g, materia }]`	Tutte le sezioni della barra dell'app (`.nav-group-panel [data-s]`), con l'etichetta del gruppo e la materia (`BSIMaterie.materiaDi` ; `'comune'` se nessuna): 197 sezioni dalla v242 (chimica 76, fisica 61, matematica 31, comuni 29). Letto dalla barra, non scritto a mano
`sezioneDi`	`(testo) → voce del catalogo | null`	La sezione che una domanda nomina: parole significative dei nomi (con radice), sigle (MS, IR, UV, NMR, MO…) pesate di più, sinonimi («spettro di massa» → MS, «infrarosso» → IR, «risonanza magnetica» → NMR, «orbitali molecolari» → MO); a parità il nome più corto. Es. «titolazioni» → `stitr` , «integrali per parti» → `smatcalc` , «ciao come stai» → `null`
`mostraSezione`	`(id) → Promise<boolean>`	Apre la sezione dietro la finestra di Spectra (`BSISpectraLive.calcolo.apriSezione` , che cambia materia se serve) e aggiorna lo schermo, così l'ologramma la mostra dal vivo
`giro`	`(materia?)`	«🎞️ Mostrami tutto»: ogni sezione della materia (`'chimica'` , `'fisica'` , `'matematica'` ; tutte se vuota) nell'ologramma, una ogni 7 s, con il contatore «n/N · nome»
`giroFerma`	`()`	Ferma il giro
`chiedi`	`(domanda, { schermo?, livello? }?) → Promise`	Manda la domanda al modello della chat; con `schermo` aggiunge il testo dello schermo e, al primo turno, l'immagine; senza `schermo` , se la domanda nomina una sezione (`sezioneDi`) diversa da quella aperta, prima la apre (`mostraSezione`) e poi chiede con lo schermo, tranne se l'utente ha tolto la spunta (`ST.schermoNo`): allora solo la domanda; le domande successive continuano la conversazione. Senza modello mostra ciò che l'app sa da sola
`schermo`	`() → { testo, tela, svg, grafici, titolo, materia }`	Legge la sezione aperta: titolo, scheda, comandi con valore e intervallo, tabelle visibili (fino a 4 × 40 righe), testo visibile, tele e SVG visibili
`immagine`	`(S) → Promise<dataURL>`	Compone in un JPEG (largo al massimo 1400 px) le tele visibili (scene `BSILab3D` ridisegnate prima) e gli SVG visibili trasformati in immagine
`sistema`	`(livello, materia) → string`	Il prompt di sistema: livello, `PROFONDITA` , struttura obbligatoria, rigore

Simbolo	Firma	Descrizione
`aggiornaSchermo`	`()`	Aggiorna la scheda dello schermo (titolo, numero di grafici e scene 3D, miniatura)
`LIVELLI`, `PROFONDITA`	array	I cinque livelli (da Scuola superiore a PhD avanzato · frontiera) e che cosa si chiede a ciascuno
`stato`	`() → { turni, livello, inCorso }`	Lo stato della conversazione

Evento **`bsi-spectra-stato`**, dalla v242 anche dalla console: `bsi-spectra-live.js` lo emette a ogni cambio di stato con `detail: { stato, dettaglio, daLive: true }` (`stato`: `'attesa'`, `'ascolta'`, `'pensa'`, `'parla'`); la figura reagisce (`'pensa'` → concentrata, gesto «pensa», mano al mento; `'ascolta'` → curiosa, «TI ASCOLTA»). La console ignora gli eventi con `daLive`: è la firma che impedisce alla console di richiamarsi all'infinito.

`window.BSIEsplora` — «⚡ Esplora con Spectra» (`bsi-spectra-esplora.js`, v242)

Simbolo	Firma	Descrizione
`CAPITOLI`	`{ chimica, fisica, matematica } →` array di 10 capitoli	Ogni capitolo: `id` (`c-…`, `f-…`, `m-…`), `liv` (0 Scuola superiore … 4 PhD avanzato · frontiera), `tit`, `sez` (l'id della sezione dell'app collegata), `scena` (la scena dell'ologramma: `orbitale`, `molecola`, `spettro`, `onda`, `pianeti`, `superficie`), `testo`, `pred: { d, o[4], g, p[4] }` (domanda, quattro risposte, indice della giusta, un perché per ognuna), `prova?: { et, min, max, passo, val }` (il cursore, in 19 capitoli su 30)
`calcola`	`(id, valore) → { v, testo } | null`	La grandezza del cursore di un capitolo: `v` il numero, `testo` la riga mostrata. Es. `calcola('c-ph', 8).v` ≈ 6,978 (HCl 10^{-8} M, con l'autoprotolisi), `calcola('f-pendolo', 1).v` ≈ 2,006 s, `calcola('m-armonica', 3).v` ≈ 7,4855; `null` per i capitoli senza cursore
`costruisci`	`(materia, pannello) → elemento`	Costruisce la pagina della materia nel pannello: testa con la barra «n/10», i dieci capitoli (`section.cap`, id `bsiEX-<id>`) con previsione, prova e i pulsanti «📺 Guardalo dal vivo», «🧠 Spiegamelo tutto», «🔊 Ascolta Spectra»; allo scorrimento il capitolo che attraversa il centro dello spazio libero sotto la figura prende la classe `attivo`, emette `bsi-spectra-dice` e porta l'ologramma alla sua scena; imposta la variabile CSS `--bsiEX-top` (l'altezza della testata fissa dell'app)
`gibbsMax`	`(N) → number`	Il massimo, calcolato numericamente su 4000 punti in (0, π/2], della somma parziale dell'onda quadra (4/π) Σ sin(kx)/k sui k dispari ≤ N (≈ 1,179 con N = 201)
`SEZIONI`	`['sesplchim', 'sesplfis', 'sesplmat']`	Le tre sezioni, registrate con `BSIFisica.sezione` (una scheda, «⚡ Esplora»)
`MEM`	`'bsi-esplora-v1'`	La chiave `localStorage` delle previsioni: `{ <id capitolo>: { giusta, quando } }`

Il modulo si carica anche in node (`require`), senza `document`: `CAPITOLI`, `calcola` e `gibbsMax` sono funzioni pure, ed è così che `test_spectra_esplora` le confronta con le formule. «🧠 Spiegamelo tutto» usa `BSISpiega.chiedi(domanda, { schermo: true, livello, nonAprire: true })` dopo `BSISpiega.mostraSezione(sez)`.

`window.BSIOloInterattivo` — l'ologramma interattivo (`bsi-spectra-olointerattivo.js`, v242)

Simbolo	Firma	Descrizione
`apri`	`(modo?, testo?)`	Apre la finestra `#bsi-olo-ov` (costruita alla prima apertura; chiama `_bsiSyncFabs`, così i pulsanti flottanti si nascondono) nella modalità data — `'spettri'`, `'universo'`, `'matematica'`, `'fisica'` — o nell'ultima usata; `testo` precompila la traccia, l'orbita o la domanda. Emette `bsi-olo-interattivo` con `{ aperto: true }`
`chiudi`	`()`	Ferma la voce, chiude la finestra ed emette `bsi-olo-interattivo` con `{ aperto: false }`; la figura condivisa torna nella sezione
`modo`	`(modo, testo?)`	Passa a un'altra modalità (scheda in alto) e rifà ologramma e passaggi
`stato`	`() → { aperto, modo, tipo, chi, MHz, passi, sel, dati }`	Lo stato: tipo di spettro, molecola o traccia (`chi`), campo NMR, titoli dei passaggi, passaggio scelto, i dati disegnati
`imposta`	`(testo)`	Scrive nel campo della modalità (traccia, domanda o orbita) e ridisegna; per matematica e fisica avvia anche la risoluzione
`scegli`	`(i, parla?)`	Sceglie il passaggio `i`, lo evidenzia nel testo e nell'ologramma; con `parla` Spectra lo dice
`SPETTRI`	oggetto	Le sei molecole di riferimento (`acetone`, `etanolo`, `acetico`, `benzene`, `toluene`, `etile`): `nome`, `formula` e i segnali `ir`, `h1`, `c13`, `ms`, `uv` — valori tipici di letteratura, non spettri misurati
`PIANETI`	array	Gli otto pianeti (scheda NASA): nome, semiasse (UA), eccentricità, periodo, massa, raggio, lune (al 2024), colore
`spettro`	`(tipo, molecola | traccia, { MHz }?) → { tipo, assi, curva, passi, segnali, righe, MHz }`	Lo spettro di una molecola o di una traccia scritta: la curva da disegnare e un passaggio per segnale (posizione, forma, attribuzione, perché); per l'^{1}H i multipletti in Hz convertiti in ppm con il campo (400 MHz se non indicato)
`pianeta`	`(nome | { a, e }) → { nome, a, e, T, q, Q, vq, vQ, passi }`	Periodo dalla terza legge (anni), perielio e afelio (UA), velocità vis-viva al perielio e all'afelio (km/s), e i passaggi (con gravità in superficie e velocità di fuga per i pianeti veri)
`leggiTraccia`	`(tipo, testo) → segnali`	Interpreta una traccia scritta: IR «1715 s, 3300 br» (s/m/w, br = larga), ^{1}H «2.17 s 6H; 1.2 t 3H J=7», ^{13}C un elenco di δ, MS «m/z intensità», UV «λ ε»; attribuzioni candidate dalle tabelle
`massa`, `insaturazione`	`(formula) → number`	Massa molare (C, H, O, N) e grado di insaturazione C − H/2 + N/2 + 1

Evento nuovo: **`bsi-olo-interattivo`** (`CustomEvent` su `document`, `detail: { aperto }`), all'apertura e alla chiusura della finestra. `bsi-spectra-figura.js` aggiunge il pulsante «⛶ Interagisci» sull'ologramma e, a finestra aperta, sposta la figura condivisa in `#bsiOI-figura`.

`window.bsiLLM` — il fornitore della chat (`bsi-ai-hub.js`, v240)

Simbolo	Firma	Descrizione
`provider`	`() → string`	Il fornitore scelto nella finestra di Spectra (o quello salvato)
`chiave`	`(provider) → string | null`	La chiave da usare per quel fornitore, come la chat; la usa «🧠 Spiega»

`window.BSIStudioLezioni` e `window.SpectraStudio` — lo studio (v239)

`BSIStudioLezioni` (`bsi-spectra-studio-lezioni.js`, usabile anche con `require` in node) espone `LEZIONI` (per ogni lezione `id`, `materia`, `predici`, `testa`, `dimostra`, `rivela`), `SCENE` (gli oggetti 3D di ogni lezione) e `calcolo`: `caduta(h, v0, g?, dt?, k?)`, `vsepr(nLegami, nDoppietti, wLP?, iter?)`, `parziale(N)`, `armonica(N)`, `coda(N)`, `fourierX2(x, N)`, `mcd(a, b)`, `coprimi(M, max, seme?)`.

`SpectraStudio`, definito dentro `spectra-studio.html`, serve ai banchi: `apri(id)`, `scegli(i)`, `testa()`, `dimostra(k)`, `rivela()`, `stato() → { lez, fase, misure, previsione, scena, volto }`.

`window.BSISpectraAndroide.crea` — il volto in una scena altrui (v239)

`crea(padre, { scena, renderer, senzaBusto }?)`: con `scena` il volto vive in una scena three.js esistente, senza renderer, luci, telecamera e ciclo di disegno propri (chi lo ospita chiama `passo(dt)` e appende `radice` al corpo); con `senzaBusto` collo e spalle del modello sono nascosti. L'oggetto espone anche `parti` (le mesh del modello per nome).

Documento aggiornato alla versione `bsi-v247`.